

Container Ecosystem Question Index - All 45 Questions

Q#	Question
Q1	Container runtimes - containerd, CRI-O, runc
Q2	OCI standards - image spec, runtime spec
Q3	Podman basics - daemonless, rootless, Docker replacement
Q4	Podman vs Docker - architecture differences
Q5	Podman pods - grouped containers, shared namespaces
Q6	Podman commands - run, build, push, ps, logs, exec
Q7	Rootless containers - user namespaces, security
Q8	Buildah basics - OCI images without daemon
Q9	Buildah vs Docker build - scripted, no Dockerfile
Q10	Skopeo - inspect, copy, sync without pulling
Q11	Docker Compose basics - multi-container apps
Q12	Compose file - services, ports, environment, depends_on
Q13	Compose networking - default network, aliases
Q14	Compose volumes - named, bind mounts, persistence
Q15	Compose commands - up, down, build, logs, exec
Q16	Podman systemd - unit files, auto-start, Quadlet
Q17	Podman in CI/CD - rootless pipelines
Q18	Podman secrets - managing without env vars
Q19	Podman networking - CNI, netavark, port forwarding
Q20	Podman remote - client-server, REST API
Q21	Buildah advanced - mount, run, copy, commit
Q22	Buildah in CI/CD - daemonless image builds
Q23	Buildah security - no daemon, no root, reduced surface
Q24	Skopeo advanced - registry copy, signing, layers

Q25	Skopeo in CI/CD - promote images between registries
Q26	Compose profiles - selective service startup
Q27	Compose override - environment-specific configs
Q28	Compose health checks - interval, timeout, retries
Q29	Compose scaling and resource limits
Q30	Compose watch - hot-reload for development
Q31	Podman Compose vs Docker Compose
Q32	Podman Quadlet - new systemd container units
Q33	Runtime comparison - containerd vs CRI-O vs Docker
Q34	Podman enterprise - pods, systemd, K8s YAML generation
Q35	Minimal images - distroless, scratch, Wolfi, Chainguard
Q36	Multi-arch builds - buildx, manifest lists, QEMU
Q37	Docker to Podman migration - strategy, pitfalls
Q38	Docker daemon down - all containers stopped
Q39	Podman rootless can't bind port 80
Q40	Compose up fails - port conflict, depends_on race
Q41	Buildah image larger than expected - optimization
Q42	Skopeo copy fails - auth, TLS, insecure registry
Q43	Compose volumes data lost after down
Q44	Podman pod containers can't communicate
Q45	Migrating docker-compose.yml to Kubernetes

SECTION 1 - Junior Core Concepts (Q1-Q15)

Q1 Container runtimes - containerd, CRI-O, runc

The Simple Answer

A container runtime is the software that actually runs containers. There are two levels: high-level runtimes manage container lifecycle (pull images, create containers, manage networking), and low-level runtimes execute the container process using Linux kernel features (namespaces, cgroups).

The Runtime Stack

runc - The low-level OCI runtime. Created by Docker, donated to the community. Takes an OCI bundle (filesystem + config.json) and creates the container process using Linux namespaces and cgroups. Every container runtime ultimately uses runc (or an alternative like crun, gVisor, Kata) to start the actual container.

containerd - High-level runtime. Originally part of Docker, now an independent CNCF graduated project. Manages: image pull/push, container lifecycle, storage (snapshots), networking (CNI). Kubernetes uses containerd via CRI (Container Runtime Interface). Docker itself uses containerd internally.

CRI-O - High-level runtime built specifically for Kubernetes. Lightweight - only what Kubernetes needs (no Docker CLI, no Docker API, no build capability). Implements the Kubernetes CRI directly. Created by Red Hat. Used in OpenShift and some Kubernetes distributions.

The Analogy - Car Engine Layers

runc is the engine - it does the actual work of running the container. containerd is the transmission - it manages the engine and connects it to the rest of the system. Docker/Podman is the dashboard - the user interface you interact with. Kubernetes talks to containerd or CRI-O (the transmission) through CRI, bypassing the dashboard entirely.

Why Docker was removed from Kubernetes: Kubernetes 1.24 removed dockershim (the Docker CRI adapter). Kubernetes now talks directly to containerd or CRI-O. Docker images still work - they are OCI-compliant. Only the Docker daemon is no longer needed. Your Dockerfiles and images are unaffected.

Q2 OCI standards - image spec, runtime spec

The Simple Answer

The Open Container Initiative (OCI) defines two standards that ensure containers are portable across tools and platforms.

Image Specification - Defines the format of container images: layers (filesystem changes), configuration (entrypoint, environment, volumes), and manifest (index of layers). Any OCI-compliant image works with Docker, Podman, containerd, CRI-O, and any OCI-compliant runtime.

Runtime Specification - Defines how to run a container: the filesystem bundle format, lifecycle operations (create, start, kill, delete), and configuration (namespaces, cgroups, mounts). runc is the reference implementation.

Why OCI Matters

Before OCI, Docker images only worked with Docker. OCI standardization means: images built with Docker work in Podman. Images built with Buildah work in Kubernetes. Any OCI-compliant tool can produce and consume images. No vendor lock-in.

Q3 Podman basics - daemonless, rootless

The Simple Answer

Podman is a container engine that runs containers without a daemon. Unlike Docker (which requires the dockerd daemon running as root), Podman runs containers as direct child processes of the user - no long-running background service. Podman is a drop-in replacement for Docker - most Docker commands work by replacing docker with podman.

Why Daemonless Matters

No single point of failure. If the Docker daemon crashes, ALL running containers stop. With Podman, each container is an independent process - one container crashing does not affect others.

Better security. The Docker daemon runs as root. Any vulnerability in the daemon gives root access to the host. Podman has no daemon - no persistent root process to attack.

systemd integration. Podman containers can be managed as systemd services directly. `systemctl start my-container` works natively. Docker containers require a wrapper.

Getting Started

`podman pull nginx:latest`. `podman run -d -p 8080:80 nginx`. `podman ps`. `podman logs container_id`. `podman exec -it container_id /bin/bash`. `podman stop container_id`. If you know Docker commands, you know Podman commands.

Q4 Podman vs Docker - architecture differences

The Simple Answer

Feature	Docker	Podman
Architecture	Client → daemon (dockerd) → containerd → runc	Direct process → conmon → runc
Daemon	Required (dockerd runs as root)	None (daemonless)
Root required	Daemon runs as root	Full rootless support
Container process	Child of daemon	Child of user (or systemd)
Daemon crash	All containers stop	Containers unaffected
Socket	<code>/var/run/docker.sock</code> (security risk)	No socket by default (API available optionally)
Compose	Docker Compose (native)	<code>podman-compose</code> or Docker Compose with Podman socket
Pod concept	No native pods	Pods (like K8s Pods)
Kubernetes	Not natively aligned	Generates K8s YAML from pods
systemd	Wrapper needed	Native integration (Quadlet)

The Process Model

Docker: You run `docker run` → Docker CLI sends request to `dockerd` (daemon) → daemon tells `containerd` → `containerd` tells `runc` → `runc` creates the container. The container is a child of the daemon, not your shell. If your terminal disconnects, the container continues (good). If the daemon crashes, the container stops (bad).

Podman: You run `podman run` → Podman directly forks `conmon` (container monitor) → `conmon` calls `runc` → `runc` creates the container. The container is a child of `conmon`, which is a child of your session (or `systemd`). No daemon in the middle. No single point of failure.

The migration path: alias `docker=podman` works for 95% of use cases. The remaining 5% involves Docker-specific features: Docker Swarm (use Kubernetes instead), Docker socket-dependent tools (configure Podman's socket compatibility), and some Compose features.

Q5 Podman pods - grouped containers

The Simple Answer

A Podman pod groups one or more containers that share the same network namespace, IPC namespace, and optionally PID namespace - exactly like a Kubernetes Pod. Containers in a pod communicate via localhost. They share the same IP address and port space.

Creating Pods

`podman pod create --name my-app -p 8080:80` creates an empty pod with port mapping. `podman run --pod my-app -d nginx` adds an nginx container to the pod. `podman run --pod my-app -d redis` adds a redis container. Now nginx can reach redis at `localhost:6379` - they share the network namespace.

Why Pods Matter

Pods bridge the gap between local development and Kubernetes. You develop with Podman pods locally, then generate Kubernetes YAML with `podman generate kube my-app > deployment.yaml`. The transition from local to Kubernetes is seamless - the same grouping, same networking model. See Q34 for enterprise patterns.

Q6 Podman commands

The Simple Answer

Podman commands are intentionally identical to Docker commands. If you know Docker, you know Podman.

Essential Commands

`podman run -d -p 8080:80 --name web nginx` - Run a container. **`podman ps`** - List running containers. `podman ps -a` includes stopped. **`podman logs web`** - View container logs. `-f` for follow. **`podman exec -it web /bin/bash`** - Interactive shell in running container. **`podman stop web`** - Stop gracefully. **`podman rm web`** - Remove stopped container. **`podman images`** - List local images. **`podman pull registry/image:tag`** - Download an image. **`podman build -t myapp:1.0`** - Build from Dockerfile. **`podman push myapp:1.0 registry/myapp:1.0`** - Push to registry. **`podman inspect web`** - Detailed container/image JSON metadata.

Pod Commands

`podman pod create --name app` - Create a pod. **`podman pod ps`** - List pods. **`podman pod stop app`** - Stop all containers in pod. **`podman pod rm app`** - Remove pod and containers. **`podman generate kube app`** - Generate Kubernetes YAML from pod.

Q7 Rootless containers - user namespaces, security

The Simple Answer

Rootless containers run entirely without root privileges. The container process runs as your regular user. Inside the container, UID 0 (root) maps to your unprivileged UID on the host. If a container escape occurs, the attacker lands as your unprivileged user - not as host root.

How User Namespaces Work

Linux user namespaces remap UIDs. Inside the container, the process thinks it is root (UID 0). On the host, it is actually your user (e.g., UID 1000). File operations, capabilities, and system calls are constrained to what your regular user can do. This is the foundation of rootless containers.

Limitations

Cannot bind to privileged ports (< 1024) by default. Cannot modify certain kernel parameters. Storage drivers are limited (fuse-overlayfs instead of native overlay). Performance is slightly lower than rootful. Network configuration requires slirp4netns or pasta (user-space networking). See scenario Q39 for the port 80 issue.

The security argument: "Why run containers as root when 99% of workloads don't need root?" Rootless containers follow the principle of least privilege. Even if the container is compromised, the attacker has no more access than a regular user. This is why Podman's rootless-first approach is considered more secure than Docker's root-daemon model.

Q8 Buildah basics - OCI images without daemon

The Simple Answer

Buildah builds OCI container images without requiring a daemon. Unlike docker build (which requires the Docker daemon), Buildah runs as a regular command-line tool. It can build from Dockerfiles OR from shell scripts - you are not limited to Dockerfile syntax.

Two Ways to Build

From Dockerfile: buildah bud -t myapp:1.0 . - Works exactly like docker build. Reads the Dockerfile, executes instructions, produces an image. "bud" stands for "build using dockerfile."

Scripted (no Dockerfile): container=\$(buildah from ubuntu:22.04) → buildah run \$container -- apt-get install -y nginx → buildah copy \$container app.conf /etc/nginx/ → buildah config --port 80 \$container → buildah commit \$container myapp:1.0. Each step is a shell command - you have the full power of bash scripting for build logic.

Why Buildah Exists

Docker build couples image building with the Docker daemon. In CI/CD pipelines, you either need Docker-in-Docker (security risk) or a shared Docker socket (bigger security risk). Buildah needs neither - it runs as a regular process. No daemon socket to mount. No privileged container needed. See Q22 for CI/CD integration.

Q9 Buildah vs Docker build

The Simple Answer

Feature	Docker build	Buildah
Daemon required	Yes (dockerd)	No
Root required	Yes (daemon is root)	No (rootless supported)
Build format	Dockerfile only	Dockerfile OR shell scripts

Layer control	Automatic per instruction	Manual (choose when to commit)
Intermediate images	Created automatically	Not created (cleaner)
CI/CD friendliness	Needs DinD or socket mount	Runs as regular command
OCI compliance	Yes	Yes
Build cache	Built-in layer cache	Supports cache with --layers

When to Use Buildah

In CI/CD pipelines where Docker daemon is not available. When you need build logic beyond Dockerfile capabilities (conditional builds, dynamic steps). When security policy prohibits the Docker daemon. When building rootless images.

When Docker build Is Fine

Local development with Docker already running. Simple, standard Dockerfiles. Teams comfortable with Docker and no security restrictions on the daemon.

Q10 Skopeo - inspect, copy, sync

The Simple Answer

Skopeo works with container images WITHOUT pulling them to local storage. It can inspect, copy, sync, and sign images directly between registries. Think of it as a registry-to-registry image management tool.

Key Commands

skopeo inspect docker://docker.io/library/nginx:latest - View image metadata (layers, architecture, labels, digest) without downloading the image. Useful for: checking image size before pulling, verifying tags, listing available architectures.

skopeo copy docker://source-registry/image:tag docker://dest-registry/image:tag - Copy an image between registries without pulling to local storage. The image goes directly from source to destination. Used for: promoting images between dev/staging/prod registries, mirroring images, cross-region replication.

skopeo sync --src docker --dest docker source-registry/repo dest-registry/ - Sync all tags of a repository between registries. Used for: registry mirroring, backup, multi-region replication.

Why Skopeo is essential in CI/CD: After CI builds and tests an image in the dev registry, Skopeo promotes it to the staging registry, then to production - without pulling 500 MB to a CI runner and pushing it back. Direct registry-to-registry copy is faster and uses less bandwidth. See Q25.

Q11 Docker Compose basics - multi-container apps

The Simple Answer

Docker Compose defines and runs multi-container applications with a single YAML file (docker-compose.yml or compose.yml). Instead of running multiple docker run commands with complex networking and volume flags, you describe your entire application stack declaratively and start it with one command.

The Analogy

docker run is like cooking one dish at a time. Docker Compose is like a full recipe book - it defines all the dishes (services), how they connect (networks), and what ingredients they share (volumes). docker compose up starts the entire meal at once.

When to Use Compose

Local development - Run your application with its database, cache, and message queue locally. **Testing** - Spin up the full stack for integration tests. **Demo environments** - One command to show the complete application. **Simple deployments** - Small-scale production on a single host (not recommended for serious production - use Kubernetes).

Q12 Compose file - services, ports, environment, depends_on

The Simple Answer

A Compose file defines services (containers), networks, and volumes.

Key Sections

services: Each service defines a container. image or build, ports, environment, volumes, depends_on, command, healthcheck, restart policy.

depends_on: Defines startup order. The web service depends on the db service - Compose starts db before web. Important: depends_on waits for the container to start, NOT for the application inside to be ready. Use depends_on with condition: service_healthy for true readiness.

environment: Set environment variables. Inline: environment: - DB_HOST=postgres. Or from file: env_file: - .env.

ports: Map host ports to container ports. "8080:80" maps host 8080 to container 80. "127.0.0.1:8080:80" binds only to localhost (not exposed externally).

restart: Restart policy. no (default), always, on-failure, unless-stopped. Use unless-stopped for production services.

Q13 Compose networking - default network, aliases

The Simple Answer

Compose automatically creates a default network for all services in the project. Services communicate using their service name as the hostname. If your Compose file has services named web and db, the web container can reach the database at db:5432. No manual network configuration needed.

Custom Networks

Define multiple networks to isolate communication. A frontend network connects web and app services. A backend network connects app and db services. The web service cannot reach db directly - it must go through app. This mirrors production network segmentation.

Aliases

A service can have multiple DNS names: networks: backend: aliases: - database - postgres. Other services on the backend network can reach it as db, database, or postgres.

Q14 Compose volumes - named volumes, bind mounts

The Simple Answer

Named volumes - Docker-managed storage. Data persists across container restarts AND docker compose down. Declared in the top-level volumes: section and referenced in services. volumes: - pgdata:/var/lib/postgresql/data. The data lives in Docker's storage directory.

Bind mounts - Map a host directory into the container. volumes: - ./src:/app/src. Changes on the host appear instantly in the container and vice versa. Essential for development - edit code on your host, see changes in the container immediately.

The Critical Difference

docker compose down removes containers and networks but preserves named volumes. docker compose down -v removes EVERYTHING including named volumes - data is gone. This is the #1 cause of accidental data loss with Compose. See scenario Q43.

Q15 Compose commands

The Simple Answer

docker compose up - Create and start all services. -d for detached (background). --build to rebuild images before starting.

docker compose down - Stop and remove containers and networks. -v also removes volumes (dangerous!). --rmi all removes images too.

docker compose build - Build/rebuild service images. --no-cache for clean rebuild.

docker compose logs - View service logs. -f for follow. service_name for specific service.

docker compose exec web bash - Run a command in a running service container.

docker compose ps - List running services with port mappings and status.

docker compose restart - Restart services without rebuilding.

docker compose pull - Pull latest images for all services.

docker compose run web npm test - Run a one-off command in a new container (for testing, migrations).

SECTION 2 - Mid-Level Production Patterns (Q16-Q32)

Q16 Podman systemd integration

The Simple Answer

Podman containers can be managed as systemd services. podman generate systemd --new --name my-container generates a unit file that creates, starts, and manages the container through systemd. The container starts on boot, restarts on failure, and is managed with standard systemctl commands.

Quadlet (Modern Approach)

Quadlet is Podman 4.4+'s native systemd integration. Instead of generating unit files, you write .container, .pod, .network, or .volume files in /etc/containers/systemd/ (root) or ~/.config/containers/systemd/ (rootless). systemd reads these files and creates the corresponding units automatically. Cleaner than generated unit files, version-controllable, and declarative. See Q32 for deep coverage.

Why systemd Integration Matters

In production without Kubernetes, you need: auto-start on boot, auto-restart on failure, dependency ordering (database starts before application), resource limits, and logging. systemd provides all of this. Podman + systemd is a production-ready container management solution for servers that do not need the complexity of Kubernetes.

Q17 Podman in CI/CD - rootless pipelines

The Simple Answer

Podman runs rootless without a daemon - perfect for CI/CD where Docker-in-Docker (DinD) is a security concern.

GitHub Actions

Use ubuntu-latest runners with Podman pre-installed. No special configuration needed. podman build, podman push work directly. No Docker socket mounting. No privileged containers. Rootless by default.

GitLab CI

Use a Podman container image as the CI runner. Or install Podman on shared runners. No need for Docker-in-Docker service. Build and push images without privilege escalation.

The Security Advantage

Docker CI/CD typically requires either: mounting /var/run/docker.sock (gives CI jobs full host control) or running Docker-in-Docker with --privileged (gives CI jobs kernel capabilities). Both are security risks. Podman needs neither - it runs as a regular process with no special privileges. This is why security-conscious organizations prefer Podman for CI/CD.

Q18 Podman secrets

The Simple Answer

Podman secrets stores sensitive data (passwords, API keys, certificates) and makes them available to containers as files - never as environment variables (which appear in process listings, logs, and inspect output).

Usage

podman secret create db_password - <<< "MySecretPass" - Create a secret. podman run --secret db_password myapp - The secret is available inside the container at /run/secrets/db_password. The application reads the file. The secret is never in environment variables, never in image layers, and never in podman inspect output.

Secrets vs Environment Variables

Environment variables are visible in: `podman inspect`, `/proc/PID/environ`, application crash dumps, logging output. Secrets mounted as files are only visible to the process that reads the file. Always use secrets for sensitive data - never environment variables.

Q19 Podman networking - CNI, netavark

The Simple Answer

Podman supports two networking backends: CNI (Container Network Interface - the legacy option) and netavark (the modern default since Podman 4.0).

netavark

Podman's native networking stack. Written in Rust. Faster than CNI. Supports: bridge networks, macvlan, custom DNS (Aardvark-dns), IPv6, and port forwarding. Default on new Podman installations.

Rootless Networking

Rootless Podman cannot create real network interfaces (requires root). Instead, it uses: **slirp4netns** - User-space TCP/IP stack. Translates container traffic. Slower but functional. **pasta** - Newer alternative to slirp4netns. Better performance. Becoming the default.

Port Forwarding in Rootless

Rootless Podman cannot bind to privileged ports (< 1024) by default. Workarounds: use `net.ipv4.ip_unprivileged_port_start=80` sysctl, use rootful Podman for specific containers needing port 80, or run behind a rootful reverse proxy (Nginx on port 80 proxying to rootless Podman on port 8080). See scenario Q39.

Q20 Podman remote - client-server, REST API

The Simple Answer

Podman can run in client-server mode. The Podman service exposes a REST API (compatible with Docker's API) on a Unix socket or TCP. A remote Podman client connects to the service - enabling management from macOS/Windows (where Podman runs in a VM) and remote server management.

Podman Machine (macOS/Windows)

`podman machine init` creates a Linux VM (using QEMU or WSL2). `podman machine start` boots the VM. The Podman CLI on macOS/Windows connects to the VM transparently. From the user's perspective, `podman run` just works.

Docker API Compatibility

Podman's REST API is largely compatible with Docker's API. Tools that use the Docker socket (`docker-compose`, `Testcontainers`, some CI tools) can work with Podman by pointing to Podman's socket: `export DOCKER_HOST=unix:///run/user/1000/podman/podman.sock`.

Q21 Buildah advanced - mount, run, copy, commit

The Scripted Build Flow

Step 1 - Create a working container: `container=$(buildah from ubuntu:22.04)`. This creates a writable container from the base image.

Step 2 - Mount the filesystem: `mnt=$(buildah mount $container)`. Now `$mnt` is a path to the container's root filesystem on the host. You can use regular host tools (`cp`, `sed`, `apt`) to modify files directly - no need for RUN instructions.

Step 3 - Run commands inside: `buildah run $container -- apt-get update && apt-get install -y nginx`. Executes in the container's namespace.

Step 4 - Copy files in: `buildah copy $container ./app /opt/app`. Copies from host to container.

Step 5 - Configure: `buildah config --port 80 --entrypoint '["/usr/sbin/nginx", "-g", "daemon off;"]' $container`.

Step 6 - Commit: `buildah commit $container myapp:1.0`. Creates the final image.

Why Scripted Builds Are Powerful

You can use if/else, loops, variables, and any host tool in the build script. Conditionally install packages based on environment. Run host-native testing tools against the image filesystem. Copy build artifacts without multi-stage builds. No Dockerfile syntax limitations.

Q22 Buildah in CI/CD - daemonless builds

The Problem with Docker in CI

Building images in CI with Docker requires either: Docker-in-Docker (docker:dind service - needs --privileged, security risk) or shared Docker socket (mounting /var/run/docker.sock - gives CI job root access to the host). Both violate the principle of least privilege.

Buildah Solution

Buildah runs as a regular unprivileged process. No daemon. No socket. No --privileged. The CI job installs Buildah and runs `buildah bud -t image:tag` - just like any other CLI tool. In rootless mode, even the build process runs without any root privileges.

GitLab CI Example

Use a Buildah container image (quay.io/buildah/stable). Build: `buildah bud -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA ..` Push: `buildah push --creds $CI_REGISTRY_USER:$CI_REGISTRY_PASSWORD $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA`. No DinD service. No privileged mode. Secure by default.

Q23 Buildah security

The Simple Answer

No daemon - No persistent root process to attack. No docker.sock to exploit. The attack surface is limited to the build process itself, which exits when done.

Rootless - Builds run as your regular user. Even if a malicious Dockerfile exploits a vulnerability during build, the attacker lands as an unprivileged user, not root.

No image cache by default - Docker's build cache can contain sensitive data from previous builds (leaked secrets in intermediate layers). Buildah does not maintain a persistent cache by default - each build is clean.

Fine-grained layer control - You choose when to commit layers. You can inspect the filesystem at any point during the build. No accidental secrets in intermediate layers because you control exactly what goes into each layer.

Q24 Skopeo advanced - registry copy, signing, layers

Registry-to-Registry Copy

`skopeo copy docker://dev-registry.example.com/app:v2.1 docker://prod-registry.example.com/app:v2.1` - Promotes an image from dev to prod without ever downloading it to a local machine. The image streams directly between registries.

Image Signing

skopeo can sign images with GPG keys or sigstore (cosign). Verify image signatures before deploying. Integrate with admission webhooks that reject unsigned images.

Layer Inspection

skopeo inspect --raw docker://nginx:latest shows the manifest - every layer hash and size. Useful for: understanding image composition, comparing images, verifying digests without pulling.

Authentication

skopeo login registry.example.com authenticates to a private registry. Credentials are stored in ~/.docker/config.json (compatible with Docker's auth format). For CI/CD: pass credentials via --creds or --authfile.

Q25 Skopeo in CI/CD - promote images

The Pattern

Build stage: Buildah builds the image, pushes to the dev registry. **Test stage:** Integration tests run against the dev image. **Promote stage:** skopeo copy promotes the tested image to staging, then production registry. No rebuild needed - the exact tested image binary is promoted.

Why This Matters

Rebuilding for each environment is risky - even with the same Dockerfile, a rebuild might pull different base image layers (if the tag has been updated). Promoting with Skopeo guarantees the exact same image binary moves through environments. What you tested is what you deploy. This is a core principle of immutable artifacts.

Q26 Compose profiles - selective startup

The Simple Answer

Profiles allow you to define optional services that only start when explicitly requested. In the Compose file: services.debug-tools.profiles: ["debug"]. Run normally: docker compose up starts only services without profiles. Run with debug tools: docker compose --profile debug up starts everything including debug services.

Use Cases

Development extras: profiler, mail catcher, mock services - only needed during development, not testing.

Database tools: pgAdmin, Redis Commander - useful for debugging, not for production. **Testing services:** selenium, test runners - only during CI test stage.

Q27 Compose override - environment-specific configs

The Simple Answer

Docker Compose automatically merges docker-compose.yml (base) with docker-compose.override.yml (local overrides). The override file adds or replaces settings from the base file.

The Pattern

docker-compose.yml - Production-like settings: specific image tags, resource limits, restart policies. **docker-compose.override.yml** - Development overrides: build from local source, mount source code as volumes, expose debug ports, disable resource limits. Git ignores the override file (.gitignore) so each developer can customize their local setup.

Multiple Override Files

`docker compose -f docker-compose.yml -f docker-compose.staging.yml up` - Explicitly specify files. The second file overrides the first. Used for: staging-specific settings, CI-specific configurations, performance testing profiles.

Q28 Compose health checks

The Simple Answer

Health checks verify that a service is actually ready to serve traffic, not just that the container started. Define in the Compose file: `healthcheck: test: ["CMD", "curl", "-f", "http://localhost:8080/health"], interval: 10s, timeout: 5s, retries: 3, start_period: 30s`.

start_period

Grace period during which health check failures do not count. Allows the application time to start up. Set to the maximum expected startup time. Without `start_period`, a slow-starting application (JVM, database initialization) appears unhealthy during startup.

depends_on with Health Checks

`depends_on: db: condition: service_healthy`. The web service waits until the db service's health check passes, not just until the container starts. This prevents "connection refused" errors during startup because the database container was running but PostgreSQL was not ready yet.

Q29 Compose scaling and resource limits

Scaling

`docker compose up --scale web=3` runs 3 instances of the web service. Each gets a unique container name and IP. The default network provides round-robin DNS - requests to `web` resolve to all 3 container IPs.

Resource Limits (deploy section)

`deploy: resources: limits: cpus: "0.5", memory: 512M, reservations: cpus: "0.25", memory: 256M`. Limits cap the maximum. Reservations guarantee the minimum. Note: the `deploy` section is primarily for Docker Swarm. For plain Compose, use `mem_limit` and `cpus` directly (or compose version 3 with `--compatibility` flag).

Q30 Compose watch - hot-reload for development

The Simple Answer

`docker compose watch` (Compose 2.22+) automatically syncs file changes from your host into running containers and rebuilds when necessary. Defined in the Compose file: `develop: watch: - action: sync, path: ./src, target: /app/src` (file changes synced instantly). `- action: rebuild, path: ./package.json` (dependency changes trigger rebuild).

Why It Replaces Volume Mounts

Volume mounts (`./src:/app/src`) work for simple cases but have performance issues on macOS/Windows (filesystem event propagation is slow). Compose watch uses efficient file synchronization. It also supports triggering rebuilds for dependency changes - something volume mounts cannot do.

Q31 Podman Compose vs Docker Compose

Options

podman-compose - Python tool that reads `docker-compose.yml` and translates to Podman commands. Works for most basic Compose files. Community-maintained. May not support all Compose features.

Docker Compose with Podman socket - Enable Podman's Docker-compatible API socket: `systemctl --user start podman.socket`. Set `DOCKER_HOST=unix:///run/user/1000/podman/podman.sock`. Run `docker compose up` - it talks to Podman through the compatible API. Better compatibility since it uses the real Docker Compose binary.

The recommendation: For full Compose compatibility, use Docker Compose with Podman's socket emulation. For simple setups, `podman-compose` works fine. For production, consider Podman pods + `systemd` (Quadlet) instead of Compose.

Q32 Podman Quadlet - new systemd container units

The Simple Answer

Quadlet is Podman 4.4+'s native `systemd` integration. Instead of generating unit files, you write declarative `.container` files that `systemd` processes directly.

Example

File `/etc/containers/systemd/webapp.container`: `[Container]`, `Image=docker.io/library/nginx:latest`, `PublishPort=8080:80`, `Volume=webdata.volume:/usr/share/nginx/html`. `[Service]`, `Restart=always`. `[Install]`, `WantedBy=multi-user.target`. Run: `systemctl daemon-reload && systemctl start webapp`. The container is now a first-class `systemd` service.

Quadlet File Types

.container - Single container. **.pod** - Pod of containers. **.network** - Container network. **.volume** - Named volume. **.kube** - Kubernetes YAML applied by Podman. All managed through `systemd` - start, stop, restart, enable, logs via `journalctl`.

Quadlet is the future of Podman production deployment. It replaces both `podman generate` `systemd` and `docker-compose` for single-host container management. Declarative, version-controllable, and natively integrated with the Linux service manager.

SECTION 3 - Senior Architecture & Advanced (Q33-Q37)

Q33 Runtime comparison - containerd vs CRI-O vs Docker

The Decision Framework

Feature	Docker	containerd	CRI-O
Scope	Full platform (build, run, manage)	High-level runtime	K8s-only runtime
CRI support	Via shim (removed in K8s 1.24)	Native CRI plugin	Built for CRI
Image build	docker build	No (use Buildah/BuildKit)	No (use Buildah)
CLI	docker CLI	ctr (low-level), nerdctl (Docker-compatible)	crictl (K8s debugging)
Resource usage	Highest (daemon + containerd + runc)	Medium (containerd + runc)	Lowest (CRI-O + runc)
Best for	Developer workstations	Kubernetes (most distributions)	Kubernetes (OpenShift, minimal)

The simple answer: Developers use Docker or Podman on their laptops. Kubernetes uses containerd (default for most distributions) or CRI-O (default for OpenShift). Nobody runs the Docker daemon on Kubernetes nodes anymore.

Q34 Podman enterprise - pods, systemd, K8s YAML

The Pattern

Define your application as a Podman pod locally. Develop and test with `podman pod create + podman run --pod`. Generate Kubernetes YAML: `podman generate kube my-pod > deployment.yaml`. Deploy to Kubernetes: `kubectl apply -f deployment.yaml`. The same application definition works locally (Podman) and in production (Kubernetes).

Production Without Kubernetes

For servers that do not need Kubernetes orchestration, Podman + Quadlet provides: container management via systemd, automatic restarts, boot-time startup, logging via journalctl, resource limits via systemd slices, and health monitoring. This is the "containers without Kubernetes" production stack.

Q35 Minimal images - distroless, scratch, Wolfi, Chainguard

The Simple Answer

Smaller images = fewer vulnerabilities, faster pulls, smaller attack surface, less storage.

scratch - Completely empty. Nothing - no shell, no libc, no files. Used for: statically compiled Go binaries. The binary IS the image. Smallest possible image (just your binary).

distroless (Google) - Base OS libraries (glibc, ca-certificates, tzdata) but no shell, no package manager, no extra tools. Available for: Java, Python, Node.js, Go, .NET. Typical size: 20-50 MB vs 100-200 MB for standard images.

Alpine - Minimal Linux distribution (5 MB base). Uses musl libc (not glibc - can cause compatibility issues). Includes shell and apk package manager. Popular but musl compatibility problems are common.

Wolfi / Chainguard - Modern minimal Linux distribution using glibc (not musl). Designed specifically for containers. Signed packages. Daily security updates. Vulnerability-free base images. Growing adoption as the "better Alpine."

The recommendation for 2025+: Wolfi/Chainguard for the best security posture. Distroless for well-supported languages. Alpine only if musl compatibility is verified. scratch for static binaries. Never use ubuntu:latest or debian:latest as base images in production - they contain hundreds of unnecessary packages and CVEs.

Q36 Multi-arch builds - buildx, manifest lists

The Simple Answer

Multi-arch images work on different CPU architectures (amd64, arm64) from a single image reference. When you pull nginx:latest on an ARM Mac (M1/M2), you automatically get the arm64 version. On an Intel server, you get amd64. This is achieved through manifest lists - an index that points to architecture-specific images.

Building Multi-Arch

docker buildx build --platform linux/amd64,linux/arm64 -t myapp:latest --push . - Builds for both architectures and pushes a manifest list. Uses QEMU emulation to build the non-native architecture (arm64 build on an amd64 host). QEMU is slower but works without physical ARM hardware.

Why It Matters

AWS Graviton (arm64) instances are 20-40% cheaper than equivalent x86 instances. Apple Silicon Macs are arm64. Kubernetes clusters may have mixed architectures. Multi-arch images let you deploy the same image reference to any node - the runtime pulls the correct architecture automatically.

Q37 Docker to Podman migration

Migration Strategy

Phase 1 - Alias. alias docker=podman on developer machines. 95% of workflows work immediately. Identify the 5% that break.

Phase 2 - Fix compatibility issues. Docker Compose: use Podman socket or switch to podman-compose. Docker socket-dependent tools (Testcontainers, dev tools): configure Podman's socket compatibility. Build processes: switch from docker build to podman build (identical syntax) or Buildah.

Phase 3 - Leverage Podman features. Rootless by default (better security). Pods for multi-container grouping. systemd/Quadlet for production. Generate Kubernetes YAML from pods.

What Breaks

Docker Swarm (no Podman equivalent - use Kubernetes). Docker Desktop GUI (Podman Desktop is the alternative). Some Docker Compose features (Swarm-specific deploy options). Tools that directly call the Docker API (need socket compatibility).

SECTION 4 - Tricky Scenarios (Q38-Q45)

Q38 Docker daemon down - all containers stopped

What Happened

The Docker daemon (dockerd) crashed or was stopped. ALL containers on the host stopped because containers are children of the daemon process. When the daemon dies, so do its children.

Step-by-Step

Step 1 - Restart the daemon. `systemctl restart docker`. Containers with `restart: always` or `restart: unless-stopped` automatically restart.

Step 2 - Check what caused the crash. `journalctl -u docker` shows daemon logs. Common causes: disk full (`/var/lib/docker`), OOM killed (daemon consuming too much memory), kernel panic, corrupted container storage.

Step 3 - If disk full, clean up: `docker system prune -a --volumes` removes unused images, containers, volumes. `docker builder prune` removes build cache.

Step 4 - If OOM, check for: runaway container consuming host memory (no memory limits set), too many containers on one host, memory leak in a container.

Prevention

This is the strongest argument for Podman. With Podman, there is no daemon - if one container crashes, others continue running. Each container is an independent process. For Docker: set memory limits on all containers, monitor daemon health, enable live-restore (keeps containers running during daemon restarts): `"live-restore": true` in `/etc/docker/daemon.json`.

Q39 Podman rootless can't bind port 80

Why It Happens

Linux restricts binding to ports below 1024 to root processes. Rootless Podman runs as your regular user - it cannot bind to port 80 or 443.

Fix Options

Option 1 - Change the unprivileged port start. `sysctl net.ipv4.ip_unprivileged_port_start=80` allows any user to bind to port 80+. Add to `/etc/sysctl.conf` for persistence. Simple but affects the entire system.

Option 2 - Use a high port + reverse proxy. Run the container on port 8080 (rootless). Run Nginx or Caddy as a system service (rootful) on port 80, proxying to `localhost:8080`. The proxy handles TLS termination and port 80/443 binding.

Option 3 - Use rootful Podman for just this container. `sudo podman run -p 80:80 nginx`. Only this specific container runs as root. Other containers remain rootless.

Option 4 - Use iptables port redirect. `iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080`. Traffic arriving at port 80 is redirected to 8080. Requires root for the iptables rule only, not for the container.

The recommended approach: Option 2 (reverse proxy) for production. It adds TLS termination, rate limiting, and caching - things you need anyway. The rootless container stays secure, and the proxy handles the privileged port binding.

Q40 Docker Compose up fails - port conflict, depends_on race

Port Conflict

Error: "Bind for 0.0.0.0:5432 failed: port is already allocated." Another container or host service is using the port. Fix: `docker ps -a | grep 5432` to find the conflicting container. Or `ss -tlnp | grep 5432` to find any process using the port. Stop the conflicting service or change the port mapping in the Compose file.

depends_on Race Condition

The web service depends_on db. Compose starts db first, then web. But PostgreSQL takes 10 seconds to initialize. Web starts at second 2, tries to connect, and crashes - PostgreSQL is not ready yet.

Fix: Use health checks with depends_on condition: depends_on: db: condition: service_healthy. Add a health check to the db service: healthcheck: test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s, timeout: 3s, retries: 5. Compose waits for the health check to pass before starting web.

Q41 Buildah image larger than expected

Step-by-Step

Step 1 - Analyze layers. buildah inspect myimage:latest shows layer sizes. Or use dive (a tool for exploring image layers).

Step 2 - Common causes. Package manager cache not cleaned: `apt-get install -y nginx && rm -rf /var/lib/apt/lists/*` must be in the SAME RUN instruction. If in separate instructions, the cache is in an earlier layer and cannot be removed. Build dependencies included: install build tools, compile, but do not remove build tools. Use multi-stage builds instead.

Step 3 - Squash layers. buildah commit --squash \$container myimage:latest combines all layers into one, removing intermediate bloat. Trade-off: no layer caching for future builds.

Step 4 - Use a minimal base image. Switch from ubuntu:22.04 (77 MB) to wolfi-base (12 MB) or distroless (20 MB). The base image is often the largest contributor to image size.

Q42 Skopeo copy fails - auth, TLS, insecure registry

Step-by-Step

Step 1 - Authentication failure. "unauthorized: authentication required." Fix: skopeo login dest-registry.example.com before copying. Or pass credentials: `--dest-creds user:password`.

Step 2 - TLS certificate error. "x509: certificate signed by unknown authority." The registry uses a self-signed certificate or a private CA. Fix (testing only): `--dest-tls-verify=false`. Fix (production): add the CA certificate to the system trust store: `cp ca.crt /etc/pki/ca-trust/source/anchors/ && update-ca-trust`.

Step 3 - Insecure registry (HTTP). "http: server gave HTTP response to HTTPS client." The registry does not support HTTPS. Fix (testing only): `--dest-tls-verify=false`. Fix (production): enable HTTPS on the registry. Never use HTTP registries in production.

Step 4 - Permission denied. The user does not have push access to the destination repository. Check registry permissions. In Harbor: verify the user's role in the project. In ECR: verify IAM permissions include `ecr:PutImage`.

Q43 Compose volumes data lost after down

What Happened

A developer ran `docker compose down -v` after testing. The `-v` flag removes all named volumes. The database data stored in a named volume is gone.

Recovery

If using named volumes with no external backup: the data is gone. `docker volume ls` shows no matching volume. Docker volumes are not automatically backed up.

If the volume still exists (docker compose down without -v): the data is safe. docker volume ls shows the volume. Recreate the containers with docker compose up - they reattach to the existing volume.

Prevention

Never use docker compose down -v without confirming the data is backed up or expendable. Create a shell alias: alias dc-clean='echo "WARNING: This removes ALL data. Type YES to confirm:" && read confirm && [["\$confirm" == "YES"]] && docker compose down -v'. Implement database backups independent of Docker volumes. For critical data, use external storage (bind mounts to a backed-up directory, or remote storage like S3).

Q44 Podman pod containers can't communicate

What Happened

Two containers in a Podman pod cannot reach each other. curl http://localhost:8080 from container A fails to reach container B which listens on port 8080.

Why It Happens

Containers in a Podman pod share the network namespace - they communicate via localhost. But the container may be listening on 127.0.0.1:8080 (IPv4 loopback) while the other container tries [::1]:8080 (IPv6 loopback) or vice versa.

Step-by-Step

Step 1 - Verify the containers are in the same pod. podman pod inspect my-pod shows member containers.

Step 2 - Check listening address. podman exec container-b ss -tlnp | grep 8080. Is it listening on 0.0.0.0:8080 (all interfaces - correct) or 127.0.0.1:8080 (IPv4 only)?

Step 3 - Check from the correct address. If container B listens on 0.0.0.0:8080, container A should reach it at localhost:8080. If listening on a specific IP, use that IP.

Step 4 - If containers are NOT in a pod (just on the same Podman network), they communicate via container names, not localhost. Check: podman network inspect to verify both containers are on the same network.

Q45 Migrating docker-compose.yml to Kubernetes

Step-by-Step

Option 1 - Podman generate kube. If using Podman: create a pod with your containers: podman pod create --name app and podman run --pod app Then: podman generate kube app > deployment.yaml. Produces Kubernetes YAML from the running pod. Review and adjust for production (add resource limits, health checks, replicas).

Option 2 - Kompose. kompose convert reads docker-compose.yml and generates Kubernetes manifests (Deployments, Services, PersistentVolumeClaims). Not production-ready - output needs significant review and customization. Good as a starting point.

Option 3 - Manual conversion. The most reliable approach. Map each Compose service to a Kubernetes Deployment + Service. Map Compose volumes to PersistentVolumeClaims. Map Compose networks to NetworkPolicies. Map Compose environment to ConfigMaps/Secrets. Map Compose health checks to Kubernetes probes.

What Does NOT Translate Directly

depends_on → Kubernetes handles startup ordering via readiness probes, not dependency declarations.
Compose build → Kubernetes does not build images - push to a registry first. Host bind mounts → Use

PersistentVolumes or ConfigMaps instead. Compose networking (auto DNS) → Kubernetes Services provide DNS automatically but with different naming.

The honest advice: Compose-to-Kubernetes conversion tools (Kompose, podman generate kube) produce a starting point, not production-ready manifests. Plan to spend 2-4 hours per service manually adjusting: resource limits, health checks, security contexts, persistent storage, secrets management, and networking.

Interview Tips

For Junior Roles

Focus on Q1-Q15. Know the difference between Docker and Podman (daemon vs daemonless). Understand OCI standards. Be able to write a basic docker-compose.yml. Know what Buildah and Skopeo do.

For Mid-Level Roles

Q1-Q32. Be ready to discuss Podman systemd integration, rootless security benefits, Buildah in CI/CD, and Compose health checks. Know when to use Compose profiles and overrides.

For Senior Roles

All 45 questions. Runtime comparison, minimal images, multi-arch builds, and migration strategy separate senior from mid-level. Be ready to design a container workflow without Docker.

Quick Reference Cheat Sheet

Topic	Key Takeaway
runc	Low-level OCI runtime. Runs the actual container. Used by everything.
containerd	High-level runtime. Default for Kubernetes. Manages images and containers.
CRI-O	K8s-only runtime. Minimal. Used by OpenShift.
OCI	Standards for images and runtimes. Ensures portability across tools.
Podman	Daemonless, rootless. Drop-in Docker replacement. alias docker=podman.
Podman pods	Group containers sharing network namespace. Like K8s Pods.
Rootless	No root needed. Container escape = unprivileged user. More secure.
Buildah	Build images without daemon. Dockerfile or shell scripts. CI/CD friendly.
Skopeo	Registry-to-registry operations. Copy, inspect, sync without pulling.
Docker Compose	Multi-container apps from YAML. up/down/logs. Dev and testing.
Compose networking	Services reach each other by name. Custom networks for isolation.
Compose volumes	Named volumes persist across down. -v flag deletes them (dangerous!).
Compose health checks	condition: service_healthy in depends_on. Use start_period.
Quadlet	Podman + systemd native integration. .container files. Production without K8s.
Distroless/Wolfi	Minimal base images. Fewer CVEs. No shell. Smaller attack surface.
Multi-arch	buildx for amd64+arm64. Manifest lists. Graviton savings.
Docker to Podman	alias docker=podman. Podman socket for Compose compatibility.
Port 80 rootless	sysctl unprivileged_port_start=80 or reverse proxy on port 80.